Eclipse IDE Cheat Sheet

Beginner’s guide on navigating through Eclipse

By Alizah Saqib and Deb Harding

Contents

3. Eclipse IDE
4. Create Project
5. Create Class and Java File
6. Navigating Java Perspective
7. Edit, Run and Compile
8. Add Existing Java File with Drag and Drop
9. Errors
10. Warnings
11. Perspective and Views
12. Formatter Profile Set Up
13. Auto-Format Save
14. Change Themes, Colors Fonts
15. Debug - Set Breakpoints
16. Debug - Start Debug Session
17. Debug - Step Through Code
18. Generate Comments
19. Keyboard Short Cuts
20. More Support

Eclipse IDE

Different IDEs can be used for different languages and use cases. [10 Best Java IDE For Developers in 2025 - GeeksforGeeks]

Why and IDE?

- Makes coding easier and faster
- Helps you avoid mistakes
- Organizes your project files in one place
- Gives you immediate feedback while you work

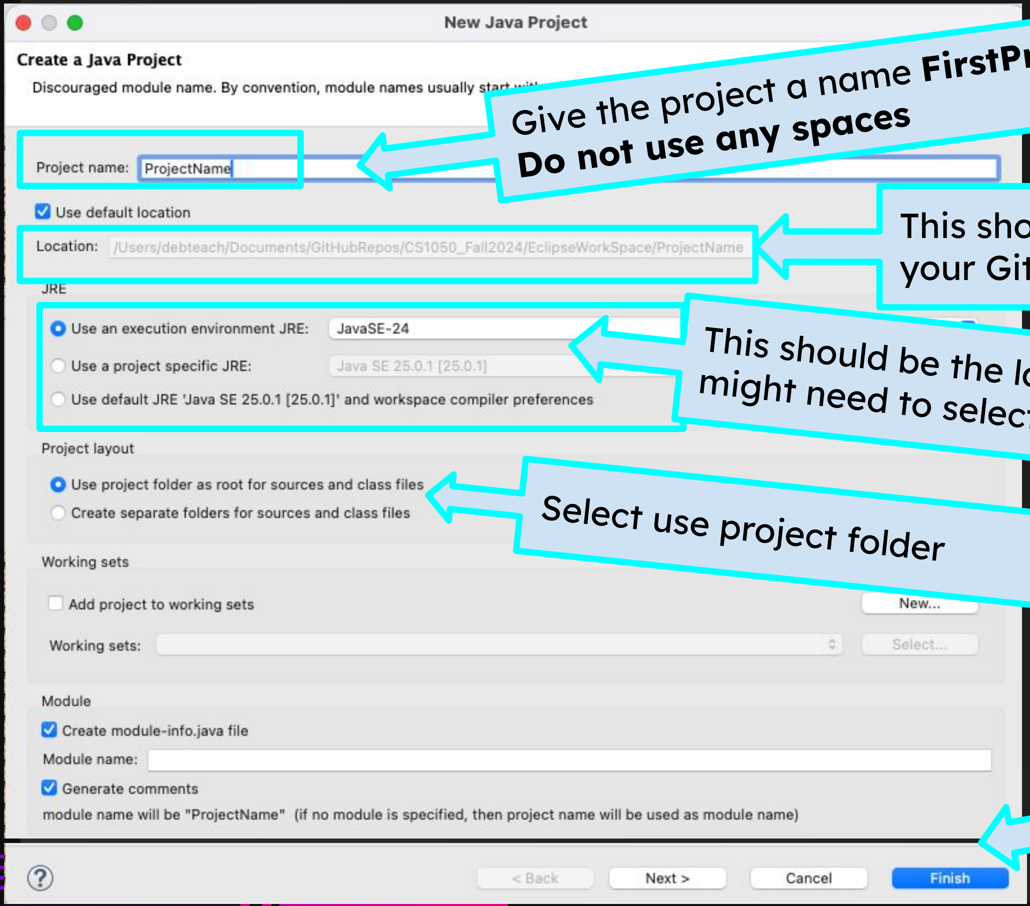
Why Eclipse?

- Free & Reliable — No cost, works across Windows, macOS, and Linux.
- Industry-Standard Tool — Still widely used in Java development.
- Java-Focused Features — Offers auto-completion, auto-formatting, debugging, and more to make coding smoother.
- Real-World Learning — Helps you practice organization, testing, and debugging in a professional environment.

First set up [Git and Github] and then [Setup Eclipse] on Windows or Mac OS

Create Project JDK-25

1. On the top menu, select **File->New-> Java Project** to display the New project dialog



Give the project a name **FirstProject**
Do not use any spaces

This should already be set to the EclipseWorkSpace in your GitHubRepos/CSJavaRepos folder

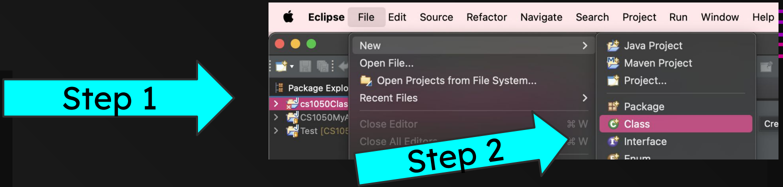
This should be the latest version you installed JavaSE25 or some might need to select Default depending on how you set it up.

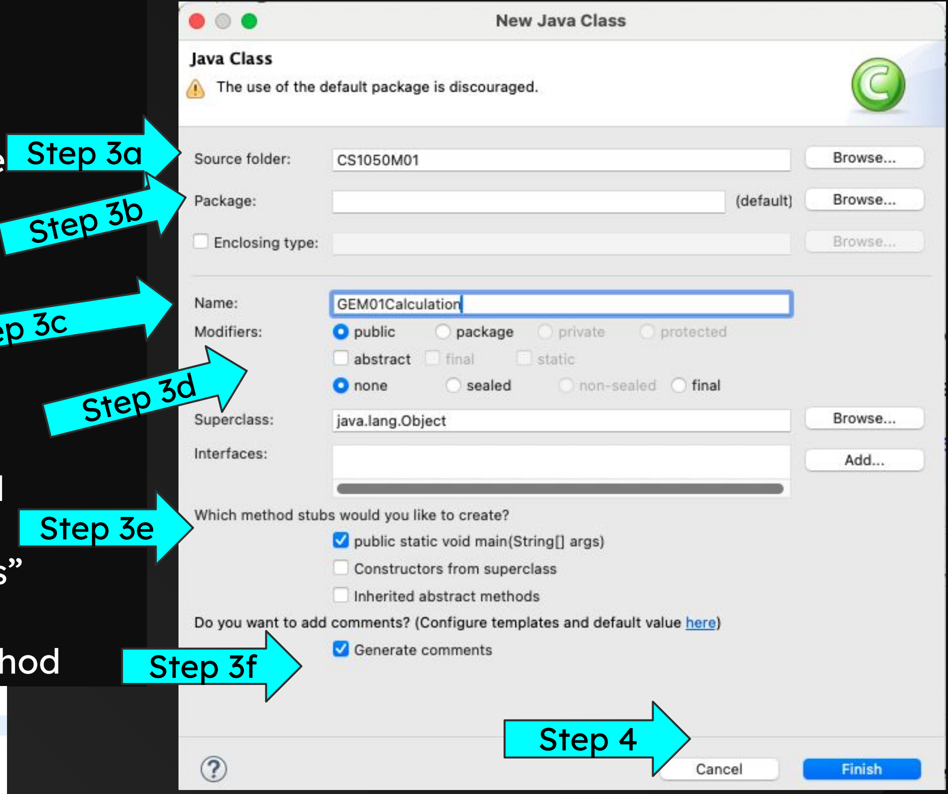
Select use project folder

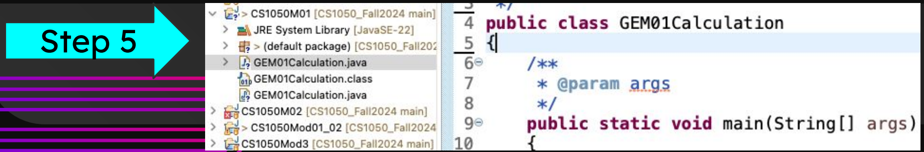
Make sure all settings are correct.
Click Finish.

Create Class & Java File

1. Select the project folder you want to create it in - in the package explorer on the left, click on your project to highlight it
2. Create a new class - click file > new > class
3. Configure settings -
   a. Source folder should be set, if not, select the project folder you want it to go in
   b. Package should be empty, if not, delete the package
   c. Enter a name for your class (e.g., GEM01Calculation)
   d. Modifiers: checkmark “public”
   e. Method stubs: checkmark “public static void main(String[] args)
   f. Comments: checkmark “generate comments”
4. Click finish to create the class
5. This create your java file with class and main method

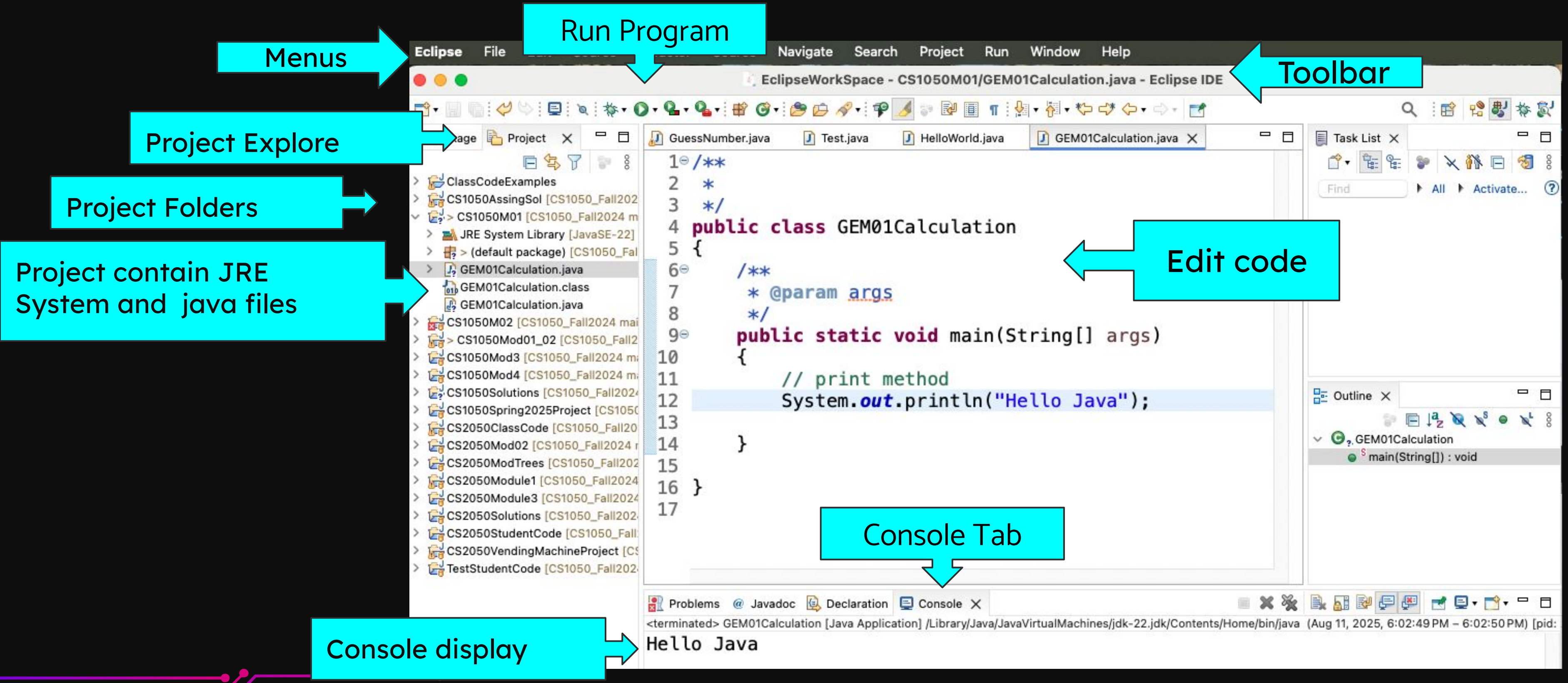






Navigating Java Perspective

Running your code



Edit, Run and Compile

When you click Run (▶) in Eclipse, two things happen:

1. Save & Compile – Eclipse automatically saves your changes and compiles your .java source files into .class files (Java bytecode).
2. Run on the JVM – The compiled .class files are then executed by the Java Virtual Machine (JVM).

Compilation checks your code for syntax errors and turns it into a format the JVM can understand. If there are errors, Eclipse won't run the program until they're fixed.

Warnings don't stop compilation, but they should still be fixed to avoid future problems.

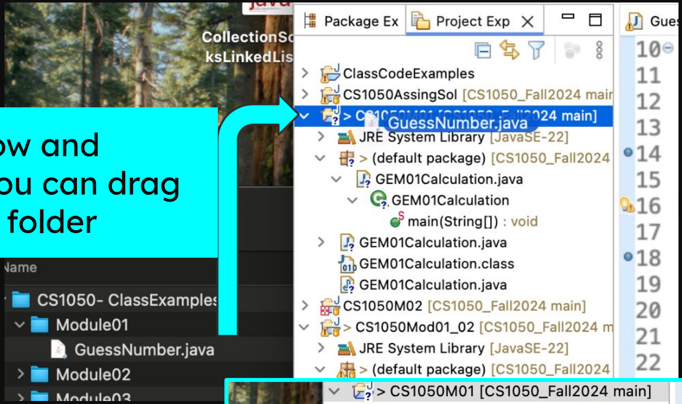
Editing Code:

- You can change your code anytime in the editor.
- When you run the program again, Eclipse automatically recompiles the changed files.
- You don't have to manually compile in most cases—Eclipse handles it for you.

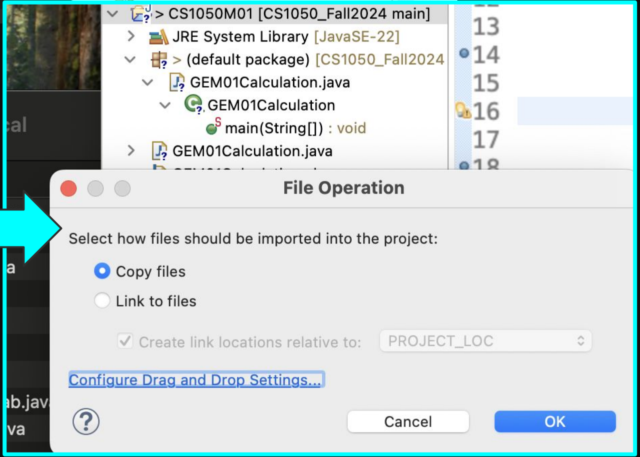
Add Existing Java File with Drag and Drop

You can add an existing Java file to a current project.

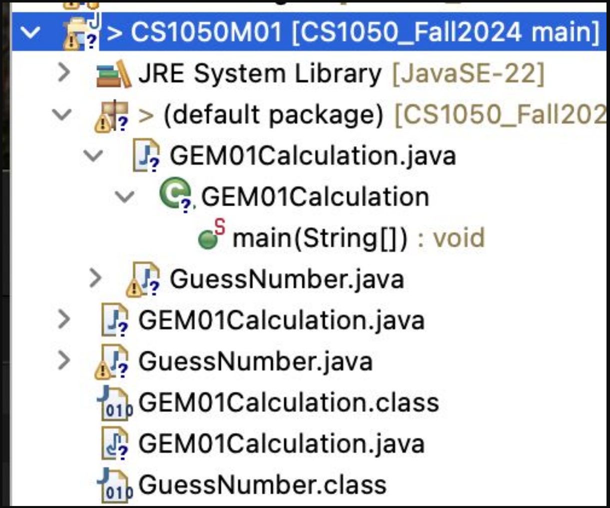
1. Make folder window and Eclipse window so you can drag java file into project folder



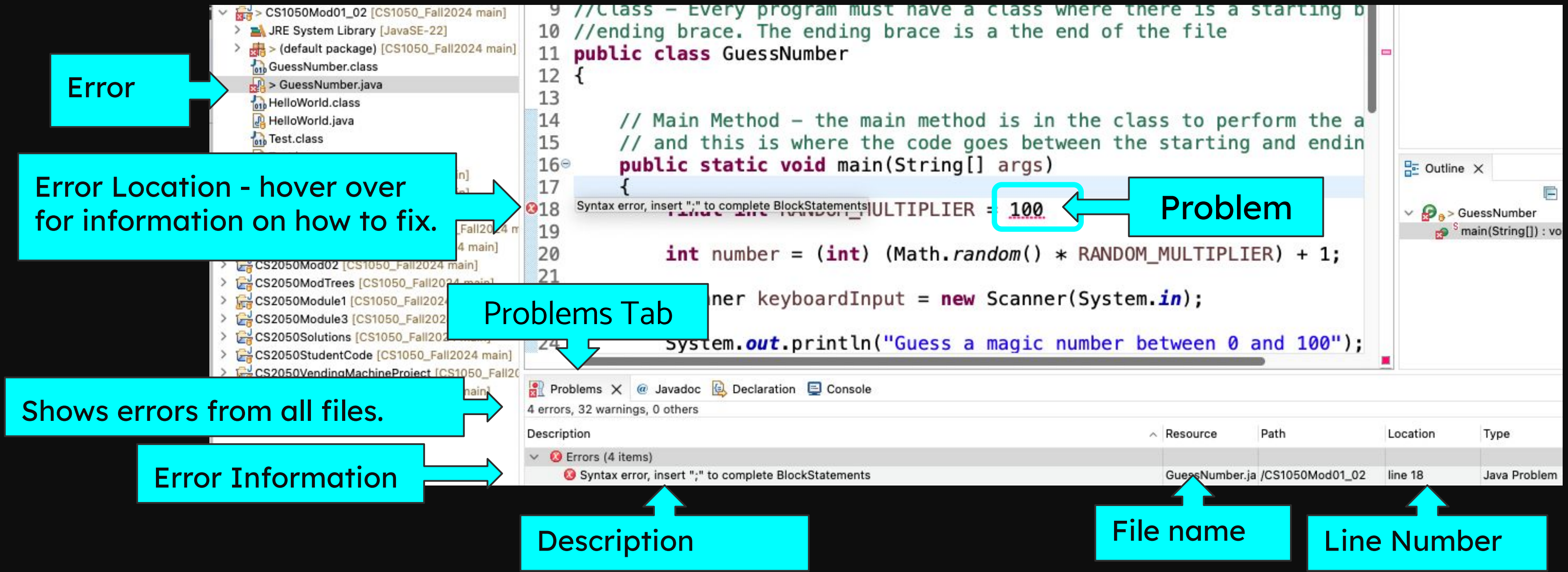
2. Click copy files so it will put the file in your repo workspace folder for versioning.



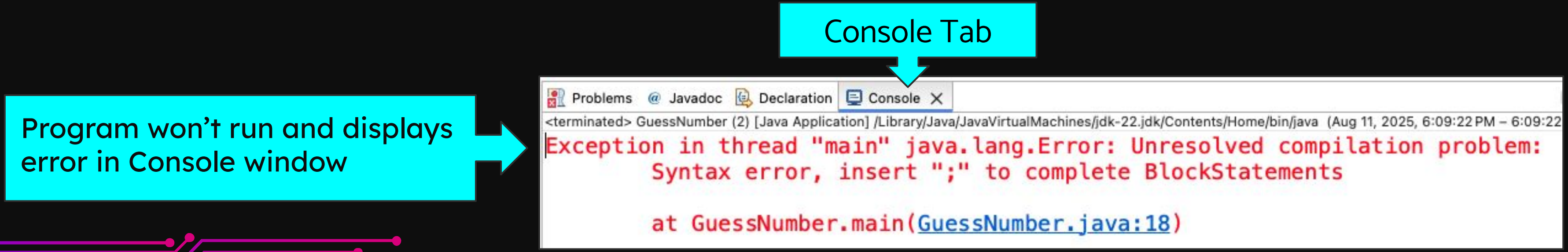
You should see the .java and .class file in the project. If you go to your github desktop you can commit and push a new version.



Errors



Error
Error Location - hover over for information on how to fix.
Problems Tab
Shows errors from all files.
Error Information
Problem
Description
File name
Line Number
CS1050Mod01_02 [CS1050_Fall2024 main]
JRE System Library [JavaSE-22]
(default package) [CS1050_Fall2024 main]
GuessNumber.class
GuessNumber.java
HelloWorld.class
HelloWorld.java
Test.class
CS2050Mod02 [CS1050_Fall2024 main]
CS2050ModTrees
CS2050Module1
CS2050Module3
CS2050Solutions
CS2050StudentCode [CS1050_Fall2024 main]
CS2050VendingMachineProject
//ending brace. The ending brace is a the end of the file
public class GuessNumber
{
// Main Method - the main method is in the class to perform the a
// and this is where the code goes between the starting and endin
public static void main(String[] args)
{
Syntax error, insert ";" to complete BlockStatements
100
int number = (int) (Math.random() * RANDOM_MULTIPLIER) + 1;
keyboardInput = new Scanner(System.in);
System.out.println("Guess a magic number between 0 and 100");
Outline
GuessNumber
main(String[]) : vo
Problems
Javadoc
Declaration
Console
4 errors, 32 warnings, 0 others
Description
Resource
Path
Location
Type
Errors (4 items)
Syntax error, insert ";" to complete BlockStatements
GuessNumber.ja
/CS1050Mod01_02
line 18
Java Problem

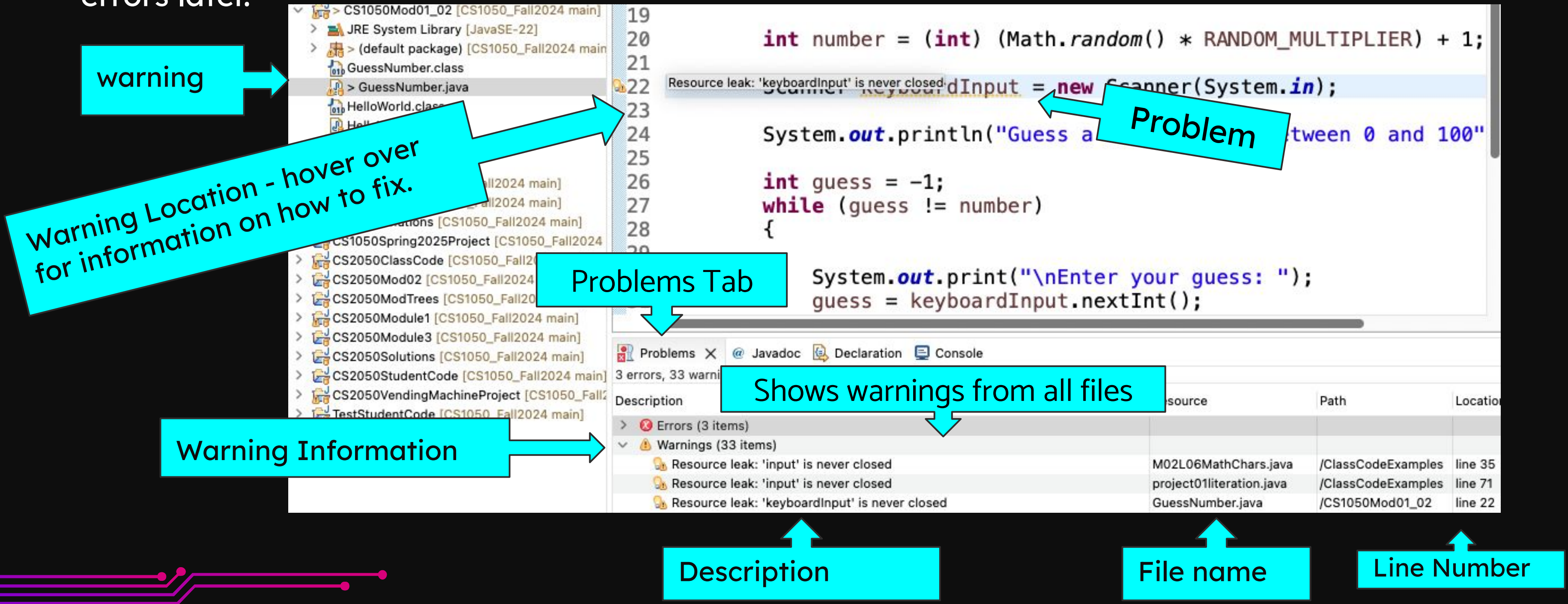


Console Tab
Program won't run and displays error in Console window
Problems
Javadoc
Declaration
Console
<terminated> GuessNumber (2) [Java Application] /Library/Java/JavaVirtualMachines/jdk-22.jdk/Contents/Home/bin/java (Aug 11, 2025, 6:09:22 PM – 6:09:22
Exception in thread "main" java.lang.Error: Unresolved compilation problem:
Syntax error, insert ";" to complete BlockStatements
at GuessNumber.main(GuessNumber.java:18)

Warnings

Your program should not contain warnings. Why fix them?

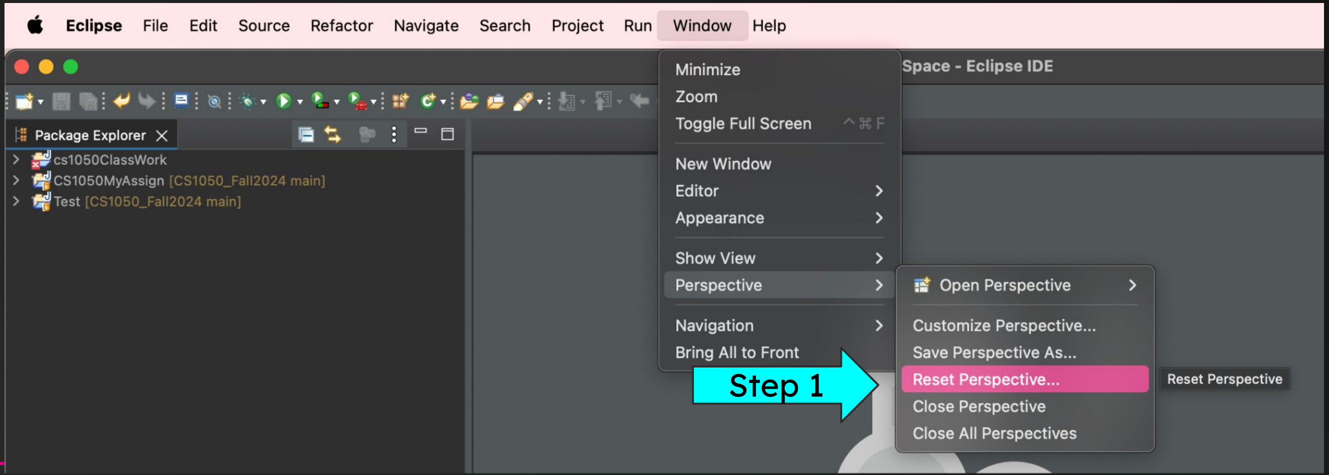
- Possible Security Risks – Some warnings indicate unsafe code that could be exploited.
- Maintainability – Clean code is easier for you (and others) to read, debug, and improve.
- Professional Standards – In industry, code reviews often require zero warnings before approval.
- Prevent Future Bugs – Warnings can signal logic issues or outdated methods that might cause errors later.



Perspective and Views

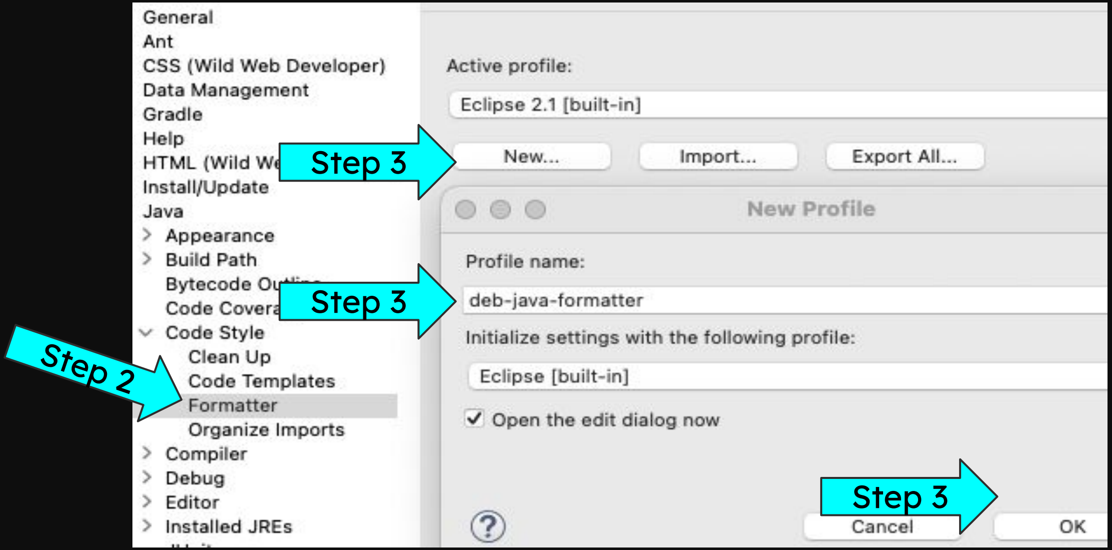
A “perspective” is just a layout of views and controls what you see in certain menus and toolbar. You will be using the Java and Debug Perspectives

1. If your “perspective” gets messed up Click window > perspective > reset perspective
   a. If you accidentally close a panel and want to bring it back
   b. If important windows like the console or package explorer disappear



Formatter Profile Set Up

1. Go to settings (or might be called preferences)
   a. Windows OS Go to menus Window -> preferences
   b. Mac OS go to menus Eclipse -> settings
2. Then navigate to Java -> Code Style -> Format
3. Click new, Give your profile a name and click ok
4. Go to the Braces section, for all categories set Position to Next line.
5. Click ok to save profile.
6. Click apply and save



This helps find error when brace is missing or incorrectly placed.

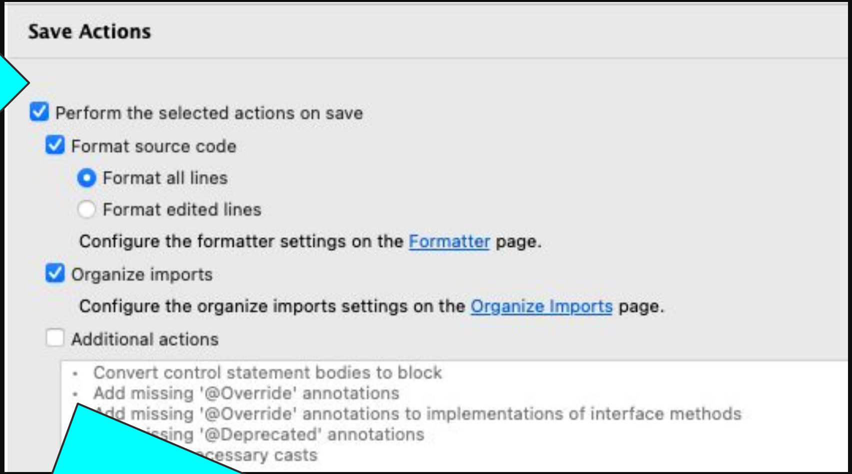
Auto-Format on Save

Save Actions on, running code will always auto-format since Eclipse saves before compiling.

1. Go to
   a. Mac: Eclipse → Settings → Java → Editor → Save Actions
   b. Windows: Window → Preferences → Java → Editor → Save Actions
2. Select these save actions

Step 1

Step 2



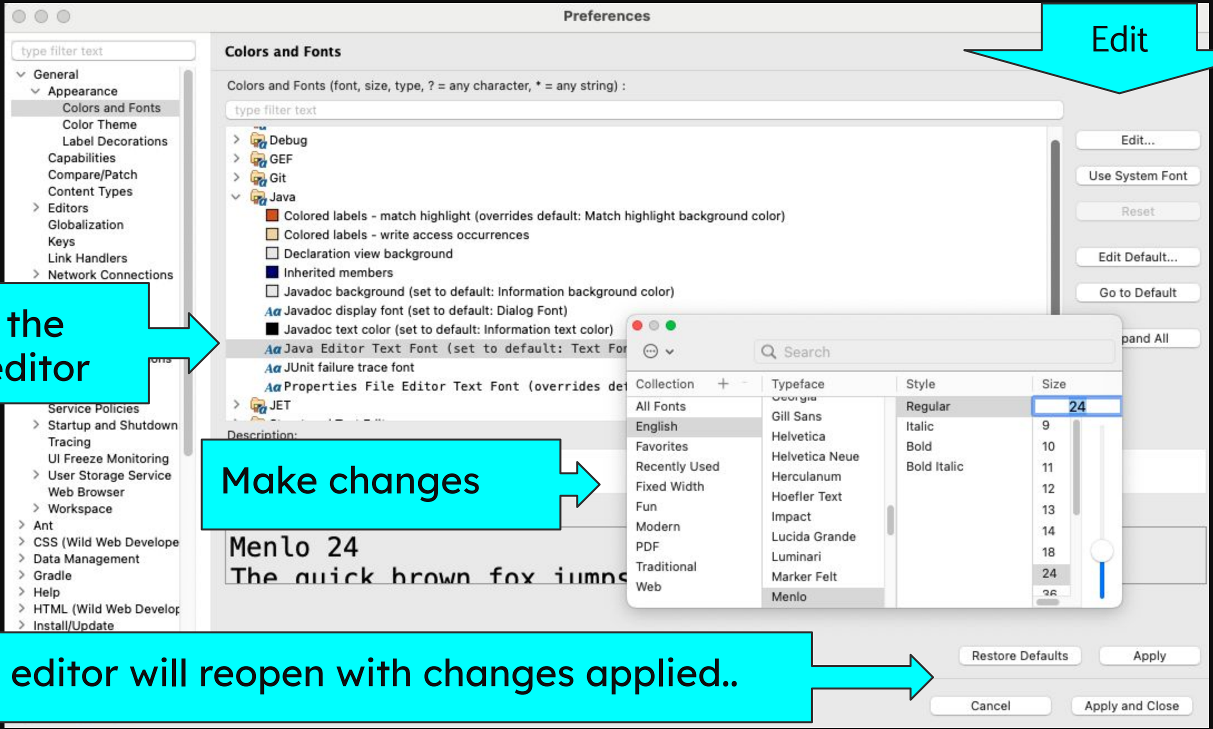
Type a messy block of code, then click Run — you should see the the correct format.

Change Themes, Colors Fonts

[6 Best Fonts for Coding to Keep Your Eyes from Eyestrain | by plabs.id]

If you want change the defaults go to Settings/Preferences general-> Appearance ->

- Colors and Fonts:
- Color Theme



Edit

For example, you can edit the font and size for the text editor

Make changes

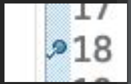
Apply and your editor will reopen with changes applied..

Debug - Set Breakpoints

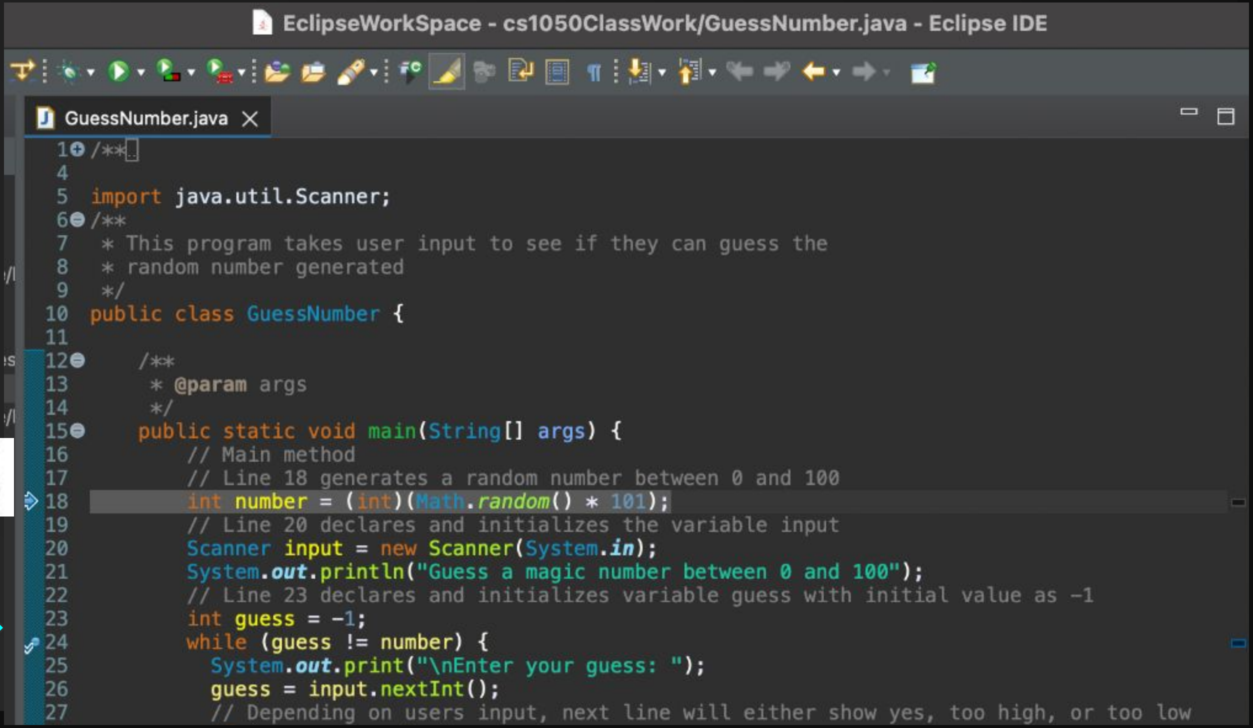
Set Breakpoints to step through each statement and see what is stored in each variables memory

a. To set a breakpoint, double click directly to the left of the line numbers
b. A blue circle will appear next to the line, indicating that a breakpoint has been set
c. To get rid of the breakpoint, double-click on it and the blue dot will disappear

Breakpoint



Breakpoint



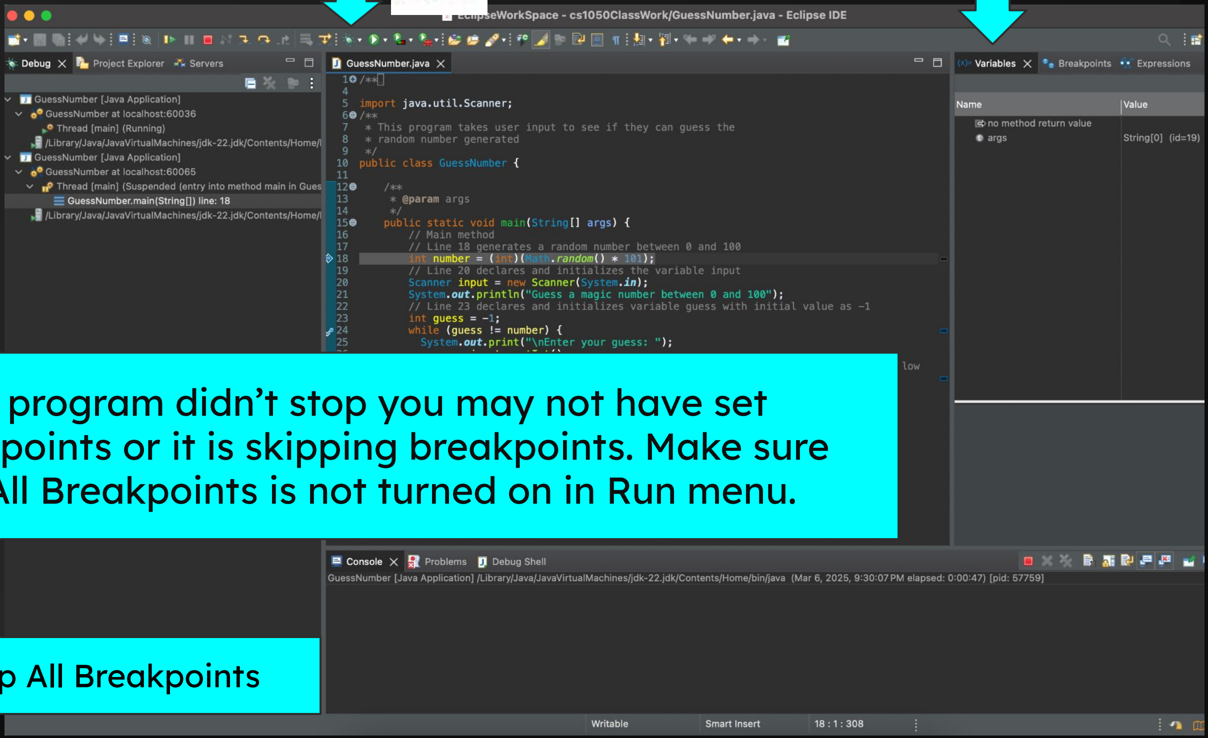
Debug: Start Debugging Session

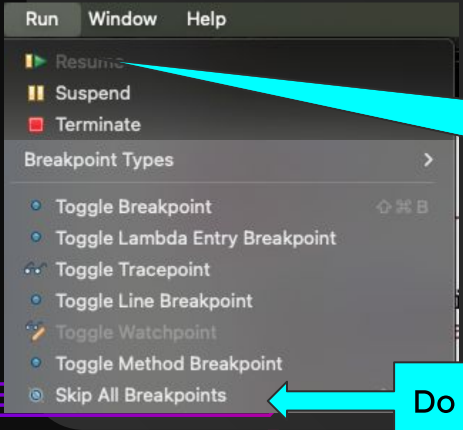
To start a debugging session click the debug icon (a green bug) in the toolbar before the run button. You should see the variables window (Debug Perspective). If you don’t this could be because

- You didn’t create your code in a project
- The java code has syntax errors

Variables window

Debug window





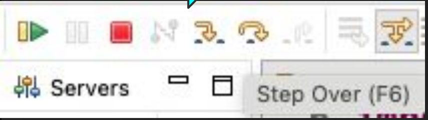
If the program didn’t stop you may not have set breakpoints or it is skipping breakpoints. Make sure Skip All Breakpoints is not turned on in Run menu.

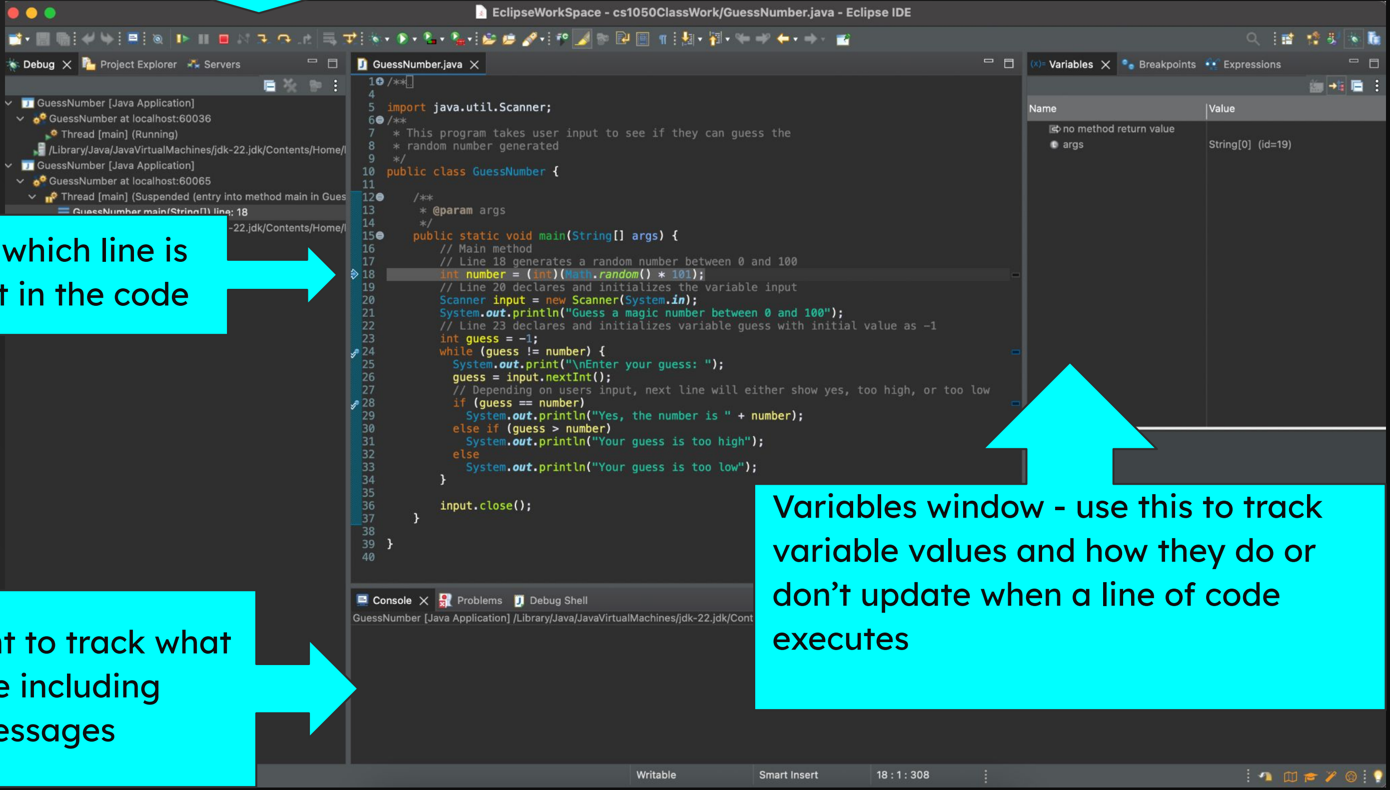
Do not select Skip All Breakpoints

Debug: Step Through Code

Use the “step over” button to control the flow of the program and move through each line manually.

Menus





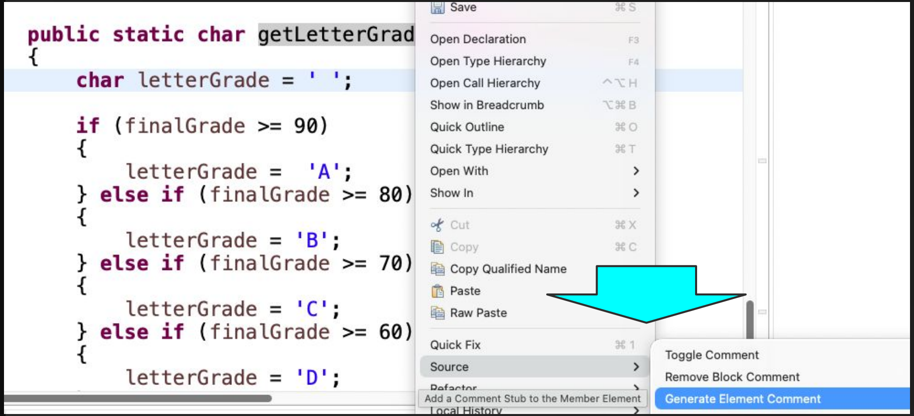
The pointer tells us which line is to be executed next in the code

Variables window - use this to track variable values and how they do or don’t update when a line of code executes

Console output - it’s important to track what is being printed to the console including print statements and error messages

Generate Comments

Right Click inside a method and select Source -> Generat ement comment




```
/**
 * @param finalGrade
 * @return
 */
public static char getLetterGrade(double finalGrade)
{
    char letterGrade = ' ';
```

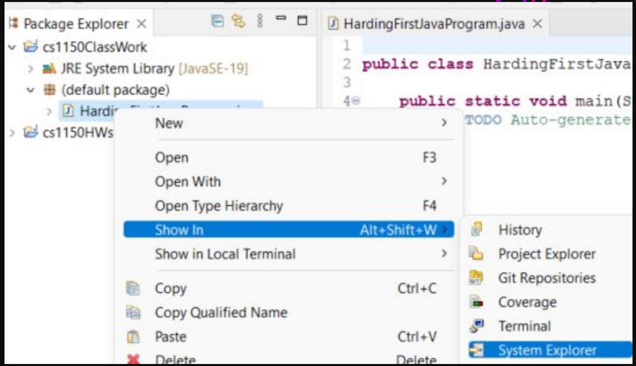
Keyboard Shortcuts

Action	Mac	Windows
Run Program	⌘ + F11	Ctrl + F11
Debug Program	F11	F11
Format Code	⌘ + ⇧ + F	Ctrl + Shift + F
Organize Imports	⌘ + ⇧ + O	Ctrl + Shift + O
Search / Find	⌘ + F	Ctrl + F
Find in Project	⌘ + H	Ctrl + H
Open Type (Class)	⌘ + ⇧ + T	Ctrl + Shift + T
Open Resource (File)	⌘ + ⇧ + R	Ctrl + Shift + R
Rename Variable/Method/Class	Alt + ⌘ + R	Alt + Shift + R
Switch Tabs	⌘ + Option + → / ←	Ctrl + PageDown / PageUp
Close Tab	⌘ + W	Ctrl + W
Undo	⌘ + Z	Ctrl + Z
Redo	⌘ + ⇧ + Z	Ctrl + Y

More Support

You can navigate to the file in your system explorer

- right click the file then go to Show In -> System Explorer



More Eclipse Resources

- [Java Debugging with Eclipse - Tutorial]
- [Eclipse Help]